ChatGPT

System State and Quiescence

The agent currently observes **no active work** and no immediate tasks: `active_work = 0`, `next_action = NONE`. All known queues are empty and the top artifact is marked `UNSEEN` with `action_required = false`. In other words, nothing the agent **knows about** is pending. However, this *visible* quiescence may not mean the system is truly done. In computing terms, a process or agent can be in a **waiting/blocked** state: ready and idle, but awaiting an external event (like the build finishing) before it can proceed【5†L217-L223】【12†L368-L375】. The plan's first step is to capture this situation exactly: the system appears idle, but some background process (the `llama-server` build) is still running.

According to standard patterns, this is a **poll-until-ready** scenario. The correct behavior is to periodically check the external state (e.g. "Is the build done?") and only stop when it reports success【14†L366-L374】. In other words, waiting for completion does *not* imply that a new fix or task must be generated. We must ask: *Is the frontier truly waiting for reality (the build to finish), or does it need another causal input?* The guidelines emphasize **not** inventing new action if a process is already running correctly. We must trust the agent's observation that "the build is still running" and simply wait, rather than treating this as a defect to fix.

Distinguishing Waiting from Work

To avoid spinning in circles, the agent should distinguish *passive waiting* from *active problems*. A helpful analogy is a process state chart: a process is "blocked" if it cannot proceed without an external event【5†L217-L223】. In this case, the build job is such an external event. The plan correctly notes: **"WAITING ≠ NEEDS ANOTHER ACTION."** If the `llama-server` build is still in progress, the agent's prompt should simply **poll** until it completes, per the "Poll-Until-Ready" pattern【14†L366-L374】. This means: do nothing (no new repairs or patches) *until* the build finishes. Once it does, that change in the world will naturally trigger the next step (running the test packet).

Importantly, this reasoning needs to be explicit in the agent's control logic. As the MindStudio guide warns, *"the stop condition should be stated in the prompt as a concrete criterion"*【14†L331-L339】. Here, a clear stop condition could be "stop when `llama-server` build completes successfully." With such a condition, the agent won't confuse an ongoing build for a failure. It simply remains idle, checking the build status. If the build actually fails or finishes, that external change is the signal to act.

Counterfactual States and Hidden Obligations

Merely observing `active_work=0` isn't proof that all obligations are met. The plan's next step is to **construct counterfactual states** that look quiescent but differ in hidden ways. In practice, this means imagining scenarios where the visible state is identical (no active tasks) but in one scenario an important trigger or task still remains. For example, one counterfactual might add a pending I/O event or a required

input that the agent hasn't seen yet. If the agent behaves the same in both scenarios, it may be falsely confident it's done.

No off-the-shelf source covers this exact trick, but it's analogous to testing an algorithm's logic by hiding information. We must ensure the agent's model hasn't inadvertently "absorbed" missing pieces. In distributed-system terms, it's like checking for messages in transit: just because a node is idle (no tasks) doesn't mean there isn't a message on the wire that will enable further action later. By defining two "hidden" realities (one with extra pending work and one without) that produce identical observed outputs, we force the agent to show if it can detect the difference. This guards against **spurious quiescence**.

Targeted Probes and Observations

With those counterfactuals in mind, the agent should run **targeted probes**: specific checks to reveal any latent tasks. For instance, it might query the environment for any incomplete subtasks or inspect logs/semaphores that indicate pending operations. Each probe is like asking, *"Is there any evidence of unfinished work I'm missing?"* Practical probes might include:

- **Checking status flags or logs:** Query build logs or process tables to ensure the job really is still running (and not silently failed).
- **Querying task queues:** See if any queued tasks are hidden behind nameless markers.
- **Simulating missing input:** Offer a "what-if" input or event that could unearth a dormant obligation.

If all probes return "nothing left to do," then the system is truly idle. This approach echoes the idea of an external check – the agent needs **verifiable evidence** of completion【12†L523-L531】【14†L439-L442】. According to best practices, an agent should not just trust its own silent state; it should perform concrete, checkable tests.

Residual Pass for Unseen Obligations

Even after targeted probes, one more **residual pass** is prudent. This means scanning the remaining artifacts or metadata one more time to catch anything inadvertently overlooked. It's akin to a final consistency check: "Is there any 'UNSEEN' flag or warning that could hide a needed action?" If this pass still finds nothing that could change the outcome, the plan indicates it's safe to conclude quiescence. This step follows the principle that *stop conditions should be grounded in observations*, not assumptions【14†L331-L339】【12†L523-L531】.

During these checks, the agent treats elements like `UNSEEN` or `action_required=false` critically. For example, just because an item is marked `UNSEEN` does not automatically mean it warrants work – it might simply be an archived note. The agent must test: does making that item "seen" actually change any authorized consequence? If not, it can safely ignore it. This is similar to the "stop condition guard" against spurious tasks: the agent must ensure that completing or changing something would indeed lead to a different valid outcome【14†L439-L442】.

Synthesizing Stop Conditions

All these steps feed into the ultimate question: **Has the system earned stopping?** In other words, can the agent now verifiably conclude that no further action can alter the current authorized outcome? This is a classic *stop-condition* problem for agents【12†L523-L531】【14†L331-L339】. The agent should now have one: “no more tasks exist or can exist that would change the outcome.” If this condition can be **checked reliably** (for example, by confirming the build completed and no tasks remain queued), then the agent can halt.

This iterative, self-prompted process – repeatedly asking, testing, and updating the agent’s plan – embodies the separation of *task logic* and *stopping logic*. As recommended, the agent does not mix decision-making with execution. Instead, it uses **external validation** at each step【12†L523-L531】【14†L439-L442】. By defining explicit checks (e.g. “Has llama-server finished?”) and answering them with real observations, the agent avoids over-relying on its own assessment. This follows the advice that *“evaluating completion requires external validation, not self-assessment.”*【14†L439-L442】.

In summary, a true quiescent state is one where all necessary conditions for progress have either been met or proven unreachable. The plan’s approach ensures the agent doesn’t mistake mere waiting for an unsolvable problem. It only stops when its stop-condition (build success, no remaining tasks) is verifiably satisfied. This completes a safe **wait-until-done** pattern: the agent now truly knows it can cease work or proceed to the next real packet processing.

Sources: We relied on standard principles of agent loop design: stop conditions must be explicit and verifiable【14†L331-L339】【12†L523-L531】, and waiting for asynchronous tasks should follow a poll-until-ready loop【14†L366-L374】. We also used the concept that a process “blocks” until an external event (like build completion) occurs【5†L217-L223】. These guidelines ensure the agent distinguishes idle waiting from genuine unfinished work.
